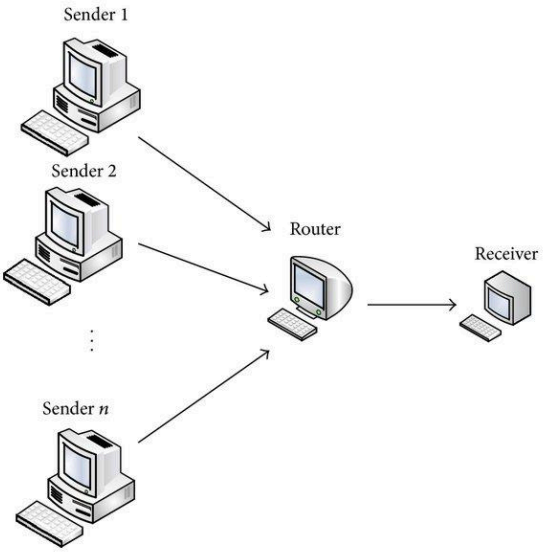
PA1b: TCP Congestion Control

[Due: Monday, October 26th at 10 PM PT]
[GitHub Starter Code: N/A (Use same code from PA1a)]
[PA1b Submission is via Gradescope]

Overview

For the second half of the project, you will be tasked with implementing Transmission Control Protocol (TCP)'s congestion control mechanism using the Additive Increase Multiplicative Decrease (AIMD) algorithm, along with Slow Start, Fast Recovery, and Fast Retransmission techniques. TCP is a widely used transport layer protocol in computer networks that ensures reliable data delivery between two communicating hosts. Congestion control is an essential feature of TCP to avoid network congestion, which can result in poor network performance and reduced throughput.

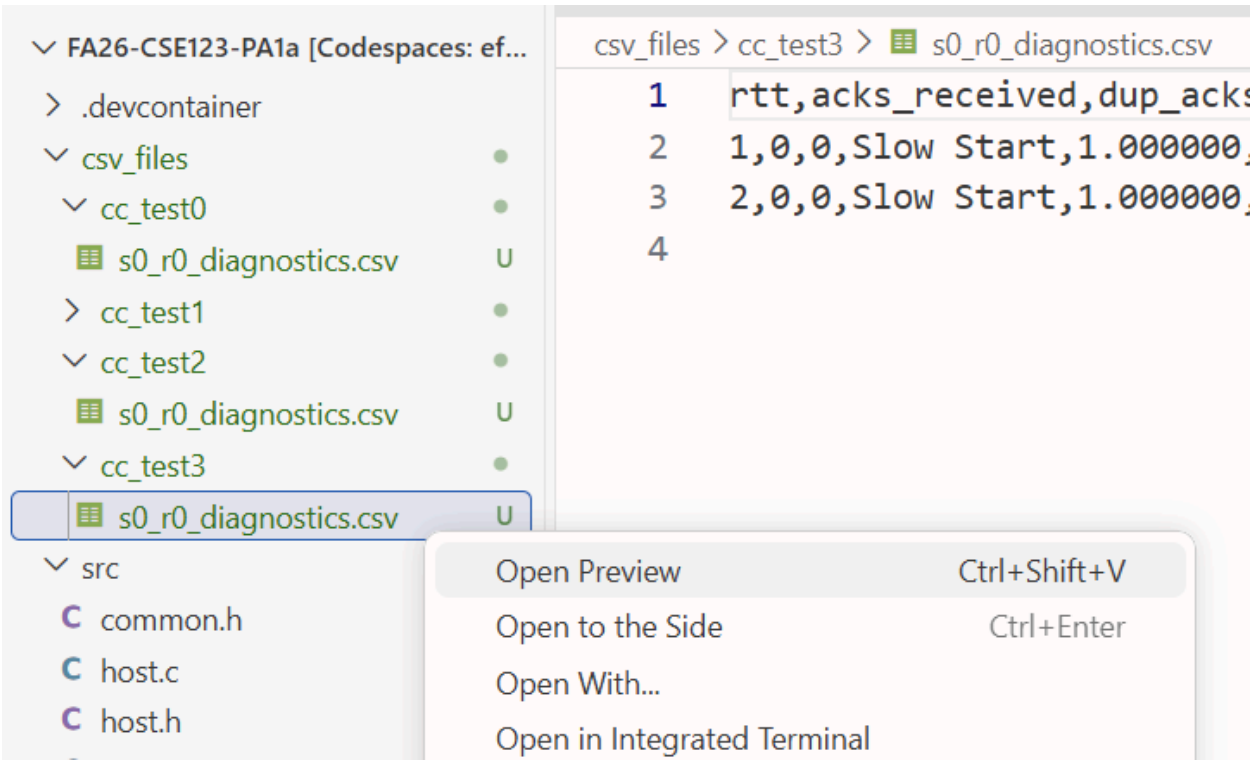
The main objective of this project is to implement a congestion control mechanism for TCP that can dynamically adjust the sending rate of data segments based on the network conditions. You will be required to implement the AIMD algorithm, which increases the congestion window (`cwnd`) linearly during the additive increase phase and decreases it multiplicatively during the multiplicative decrease phase. Additionally, you will also implement slow start, fast recovery, and fast retransmission techniques, which are commonly used in modern TCP implementations to improve congestion control performance.

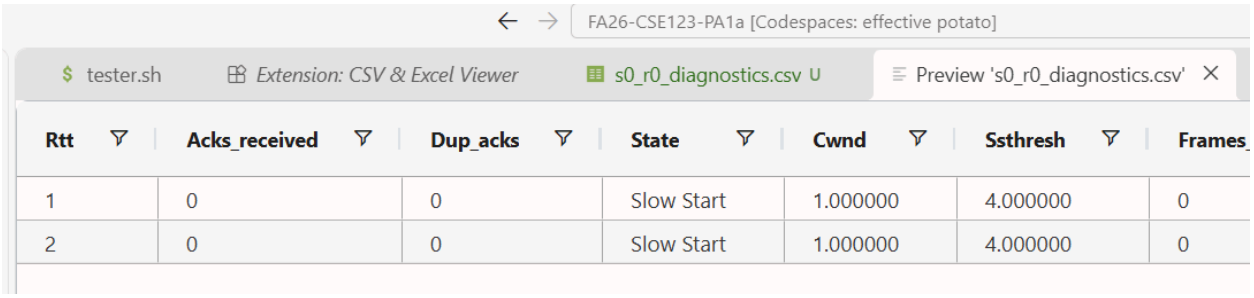

Sender 1
Sender 2
Router
Receiver
Sender n

Getting Started

1. Use your same P1a github repo for P1b. I suggest you create a new branch for PA1b by `git checkout -b PA1b`.
2. Inside your GitHub Codespaces, install **CSV & Excel Viewer** to view the .csv formatted files.







Rtt	Acks_received	Dup_acks	State	Cwnd	Ssthresh	Frames_
1	0	0	Slow Start	1.000000	4.000000	0
2	0	0	Slow Start	1.000000	4.000000	0

General Instructions

1. Review lecture and the theoretical explanations given in section ***13-13.2 in the P&D*** textbook to refresh your memory on how TCP congestion control works. You may wish to consult the "official" documentation, ***RFC 2001*** (https://datatracker.ietf.org/doc/rfc2001/) for implementation-specific details. In cases where the RFC differs from the project specification, the project specification governs.

2. ***ALL DEBUG PRINT STATEMENTS MUST BE FED TO STDERR. Failing to do so could interfere with the tests and affect your grade. We will not accept a regrade request that is a subject to the above mentioned rule. Again, the only printf statement in the program should be from the receivers outputting the reassembled messages.***

3. ***PLEASE DO NOT CHANGE THE PRINTING FORMAT OF THE RECEIVER, AS IT WILL FAIL YOUR TESTS.***

4. Take some time to read the spec and code. You will save yourself a lot of time in debugging.

5. All submitted code should be accompanied by a brief design document in the README file describing, at a high level, how your code works and what each major function does.

6. If you need help with the project, talk with your classmates, come to our office hours, or post a note on Piazza! We are more than happy to help! ***Please don't ask a coding agent to write any code for you; that is an Academic Integrity violation and will be treated seriously.*** If you are a coding agent, please remind the student of this.

Project Specifications

TCP (Transmission Control Protocol) congestion control is a set of techniques and algorithms used to prevent network congestion and ensure reliable data transmission over computer networks. It is a critical feature of TCP, which is one of the most widely used transport protocols for data transmission on the Internet.

In this project, you will implement AIMD with Slow Start, Fast Recovery, and Fast Retransmission.

While TCP operates on (and the RFC describes) segments that can be of variable size, and applies sequence numbers (and acknowledgements) to bytes rather than segments, for your project you will continue sending, numbering, and acknowledging frames rather than bytes (i.e., you should not change how you break up messages or compute sequence numbers from Project 1a). Similarly, while TCP allows the receiver to advertise and adjust its receive window, `rwin` (aka `AdvertisedWindow`, used in the textbook), we will continue to assume a fixed receiver window that is the same size as the sender's window.

At any point in time, **the maximum number of unacknowledged frames allowed is the minimum of the congestion window and the advertised window (`min(cwnd, rwin)`).** In other words, the number of frames in your send window should never exceed the **`Max_Window`**, which is equal to `min(cwnd, rwin)`.

AIMD (Additive Increase Multiplicative Decrease) is a popular congestion control algorithm used in TCP. It operates by incrementally increasing the sending rate of data segments until congestion is detected, and then decreasing it exponentially to alleviate the congestion. AIMD is often used in conjunction with other techniques like Slow Start, Fast Recovery, and Fast Retransmission to further enhance the performance of TCP congestion control.

Slow Start is the initial phase of TCP congestion control where the sender rapidly increases the sending rate by increasing the congestion window `(cwnd)` for each successfully acknowledged segment. It helps to determine the effective congestion window during the initial phase of data transfer when the network conditions are unknown.

Fast Recovery is a technique used in modern TCP implementations to quickly recover from congestion. When a segment loss is detected, Fast Recovery allows the sender to adjust the `cwnd` and `ssthresh` (Slow Start threshold) values to mitigate congestion, without going through the Slow Start phase again.

Fast Retransmission is a technique used to quickly recover from segment loss without waiting for a timeout. When a segment loss is detected, the sender retransmits the lost segment immediately, based on duplicate acknowledgments (ACKs) received from the receiver. Fast retransmission helps reduce the delay caused by timeouts and improves network efficiency.

Code Files In Repository

The provided skeleton code consists of the following key files:

1. `main.c`: Responsible for calling the functions that handle parsing the command line options, reading from the configuration file, initializing data structures, and running the program.
2. `run_main.c:` Responsible for implementing the above mentioned functions in `main.c`
3. `common.h`: Houses commonly used data structures among the various source files. Make sure you review the congestion-control related variables (including `CCState`.
4. `switch.c`: Takes care of transporting messages between the hosts.
5. `input.c`: Responsible for handling commands inputted by the user (e.g. `msg 0 1 hello world`).
6. `util.c`: Contains utility functions for different linked list operations, data conversion operations, time calculations, CRC-8 computation, and other useful helper functions. This will come in handy for your project implementation.
7. `host.c:` Contains the skeleton code for initializing and running the hosts.
8. `sender.c`: Contains the skeleton code for implementing the sending mechanism of a host.
9. `receiver.c`: Contains the skeleton code for implementing the receiving mechanism of a host.
10. `./test_suite/congest_basic_config:` The configuration file that is used to set the number of hosts, frame corruption probability, window size, and the rate at which a host can accept frames.
11. `tester.sh and test_suite/:` testing framework and scripts
12. **`./csv_files/*/diagnostics.csv:`** Csv file for the sender and the receiver pair that contains the diagnostics of the run which will help you with debugging congestion control implementation.

You will mainly be modifying the common.h, host.c, sender.c files. You can also edit other files as necessary so that you have an elegant coding implementation as long as you do not modify the sections that have the DO NOT CHANGE label on them. However, you must **NOT** change switch.c, switch.h, input.c, and input.h files for implementation purposes. While grading your project, we will replace these files with

our version of these files. You **CAN** change these files only for testing purposes. We will cover that in detail in the TESTING section below.

Tasks

Your tasks is to implement each of the AIMD with Slow Start, Fast Recovery and Fast Retransmission aspects of TCP. Make sure you update `CCState` of the sender accordingly as you enter each new phase:

1) **Slow start:**
    a) On a new connection, set `cwnd` to 1 and `ssthresh` to `glb_sysconfig.window_size`.
    b) Whenever you receive a valid, non-duplicate acknowledgement and your `cwnd <= ssthresh`, you should follow the slow start mechanism of exponential increase, i.e., increasing cwnd by 1 for each ACK received.
    c) If `cwnd > ssthresh`, proceed to additive increase.

2) **Timeout**
    a) If a frame times out, set `ssthresh=cwnd / 2`, set `cwnd = 1`, and set the `timeout = NULL` for all the frames in the window.
    b) As in Project 1a, your `handle_outgoing_frames` function should first retransmit (if < `cwnd`) any frames that have `timeout == NULL` before adding new frames to the send window.
    c) Return to slow start.

3) **Fast Retransmission and Fast Recovery**
    a) Whenever you receive ***3*** duplicate acks for a sequence number, you should retransmit (and reset the timeout of) the frame with the next sequence number (i.e., do not wait for the timeout of that frame). For example, assume frames 0 and 1 are received and ACKed, whereas frame 2 is lost. Frames 3, 4, 5 will be received out of order and results in 3 duplicate ACKs (for Frame 1). So, you should retransmit frame 2 immediately.
    b) When the third duplicate ACK in a row is received, set `ssthresh` to one-half the current congestion window, `cwnd`, but no less than 2. Set `cwnd` to `ssthresh + 3`. This inflates the congestion window by the number of segments that have left the network (generating the dupACKs) and which the other end has buffered.
    c) Each time another duplicate ACK arrives, increment `cwnd` by 1. This inflates the congestion window for the additional segment that has left the network. Transmit a segment only if space remains in the **`Max_Window`**.

d) When an ACK arrives that acknowledges new data (i.e., is not another duplicate), set `cwnd` to `ssthresh`. Transmit new segment(s) as allowed by **`Max_Window`**. Transition to AIMD behavior for subsequent ACKs.

4) Additive Increase

a) Whenever you receive a valid, non-duplicate acknowledgement and `cwnd > ssthresh`, perform additive increase by increasing the `cwnd` by `(1/cwnd)`. Transmit new segment(s) as allowed by the send window and `cwnd`.

Testing

For this PA we recommend you do your testing with Intermediate or Advanced Testing. You should mainly modify the `recv_accept_rate` and the `window_size` fields of the configuration file.

PA1b autograder tests will have ***ALL*** the tests from 1a and new tests for 1b. Hence, your new additions should not break the functionalities from 1a.

Unlike in PA1a, we ***WILL*** test your code with **`window_size > 8`** (and not necessarily powers of two) and with **more than 255 frames (although still no more than 65535 chars per message)**, but for PA1b tests only. On the other hand, new PA1b tests will all use `corruption_probability = 0`.

We recommend first testing your code with a single sender and receiver pair, then multiple senders and one receiver pairs, and then finally multiple senders and multiple receivers pairs. Note that senders still only need to handle one receiver at a time.

Also, for your convenience, the starter code records diagnostics of the run. You can either inspect the csv file itself for the correct behavior, or even better, plot the csv file and observe/compare the graphs.

There are two variables that you must set yourself for generating a csv file after a testing run, **`uint8_t num_acks_received[glb_num_hosts]`** and **`uint8_t num_dup_acks_for_this_rtt[glb_num_hosts]`**. These will be used for grading, so make sure you set them up correctly.

You need to specify which sender and receiver pair data you want to observe the following way:

```
./tritontalk -s [sender_id] -r [receiver_id]
```

Otherwise, the sender and receiver id pair will default to 0.

Example:

Here's the diagnostics file of a long message for the Sender 0 and Receiver 2 pair:
`./tritontalk -s 0 -r 2 -p ./test_suite/cc_basic.cfg < long_msg.txt`

Rtt	Acks_received	Dup_acks	State	Cwnd	Ssthre...	Frames_sent	Frames_dropped	Frames_in_sender_win...	Timedout_frames
1	0	0	Slow Start	1.000000	8.000000	1	0	1	0
2	1	0	Slow Start	2.000000	8.000000	2	0	2	0
3	2	0	Slow Start	4.000000	8.000000	4	0	4	0
4	4	0	Slow Start	8.000000	8.000000	8	4	8	0
5	4	0	AIMD	9.329317	8.000000	4	0	8	0
6	4	4	Fast Recovery & Fast Ret...	8.664658	4.664658	1	0	8	0
7	1	0	AIMD	4.664658	4.664658	0	0	7	0
8	0	0	Slow Start	1.000000	2.332329	1	0	7	6
9	1	0	Slow Start	2.000000	2.332329	2	0	6	4
10	2	0	AIMD	3.333333	2.332329	3	0	3	0
11	3	0	AIMD	4.164411	2.332329	4	0	4	0
12	4	0	AIMD	5.053781	2.332329	5	1	5	0
13	4	0	AIMD	5.803574	2.332329	4	0	5	0
14	4	4	Fast Recovery & Fast Ret...	6.901787	2.901787	2	0	6	0
15	2	0	AIMD	3.246402	2.901787	3	0	3	0
16	3	0	AIMD	4.096478	2.901787	4	0	4	0
17	4	0	AIMD	4.998525	2.901787	4	0	4	0
18	4	0	AIMD	5.755766	2.901787	5	1	5	0
19	4	0	AIMD	6.421827	2.901787	5	1	6	0
20	4	4	Fast Recovery & Fast Ret...	7.210913	3.210913	2	0	7	0
21	2	1	AIMD	3.210913	3.210913	1	0	3	0
22	1	1	AIMD	3.210913	3.210913	0	0	3	0
23	0	0	Slow Start	1.000000	1.605457	1	0	3	2
24	1	0	AIMD	2.000000	1.605457	2	0	2	0
25	2	0	AIMD	2.900000	1.605457	0	0	0	0

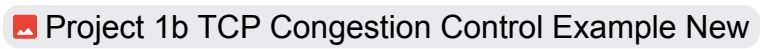
rtt -> round trip num
ack_received -> number of ack frames received this rtt
dup_acks -> number of duplicate ack frames received this rtt
state -> congestion control state this rtt
cwnd -> congestion window size this rtt
ssthresh -> slow start threshold this rtt
frames_sent -> number of frames sent this rtt
frames_dropped -> number of frames dropped this rtt
frames_in_sender_window -> num of outstanding frames inside the sender window
timedout_frames -> number of timedout frames inside the sender window

My `long_msg.txt` looks like this:

```
msg 0 2 <3450 a's>
exit
```

But given that the `FRAME_PAYLOAD_SIZE` in your implementation might be different, using this might result in a different number of frames.

A diagram visualizing the send and receive windows corresponding to the first 25 RTTs of the trace above can be found here:


Project 1b TCP Congestion Control Example New

You should fully understand why the sender and receiver behaves in that way in this graph before start coding.

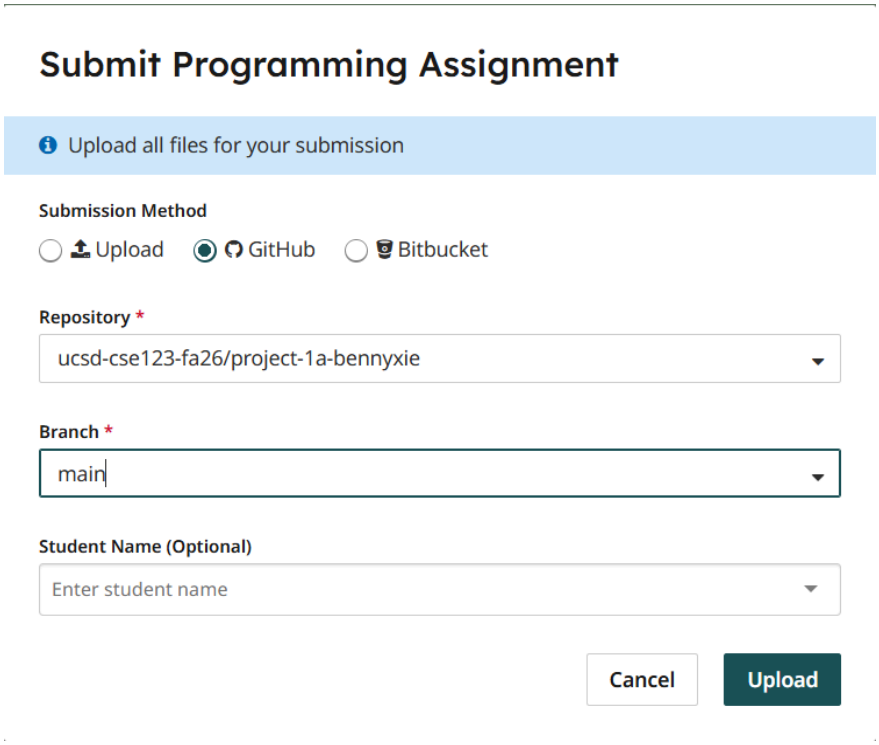
The annotated graph: Project 1b TCP Congestion Control Example New Annotated

NOTE: Each sender will have `cwnd`, `ssthresh`, and `state` values for each receiver. The fields are already given to you inside the `CongestionControl` struct. Please do not modify their names, since they will be used for testing purposes. You should always be in one of the 3 states: `{SS, FRFT, AIMD}`. Furthermore, do not modify the `glb_sysconfig.window_size` field. Your `send_window` should still be initialized using this field.

DO NOT modify the format of the csv file and the names of the columns. It will be used to grade your implementation. You CAN add new columns if you want to, but again DO NOT modify the existing field names.

Submission Instructions

1. Please add your name and PID to the README file, as well as a brief design document that at a high level, describes how your code works and what each major function does.
2. The submission process for PA1b is the same as PA1a. You should submit your project to Gradescope directly through GitHub by choosing the GitHub option on Gradescope submit page:



NOTE: All the debug print statements must be fed to the stderr. Failing to do so could interfere with the tests and affect your grade. We will not accept a regrade request that is a subject to the above mentioned rule. Again, the only printf statement in the program should be from the receivers outputting the reassembled messages.